Mapeo Hibernate

Desarrollo Entorno Servidor

Mapear Entidades

La función de Hibernate es la de relacionar nuestras clases con los objetos almenados en la base de datos en formas de tablas con columnas.

Para ello nosotros teníamos nuestros POJOs que eran clases que solo contaban con sus atributos, constructores, getters y setters.

Posteriormente para trabajar con Hibernate debíamos introducir:

- Tener un constructor público sin ningún tipo de argumentos.
- Método Getter y Setter para cada propiedad que queramos.
- La anotación **@Entity** encima de la clase.
- Implementar el interfaz **Serializable**

Mapear Entidades

Cumpliendo estas restricciones al entrar en las clases no encontrabamos con varias anotaciones que a continuación analizaremos.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

private static final long serialVersionUID = -8668594760203621162L;
@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@Column(name = "NICKNAME")
private String username;

@Column(name = "NOMBRE")
private String nombreProfesor;

@Column(name = "APELLIDOS")
private String apellidosProfesor;
```

Entidades

- Una entidad representa una clase Java (POJO) que vamos a hacer persistente
- Utilizamos la anotación **@Entity** para marcar una clase como persistente
- **@Table** nos permite definir en que tabla se hará persistente y restricciones de unicidad.
  - Si no se incluye la anotación **@Table** se considera que la tabla se llama igual que el nombre de la clase

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {
```

Identificadores

- El identificador declara el atributo que representa la clave primaria de la tabla
- Se define con la anotación **@Id**
  - Para indicar una clave primaria autoincremental utilizamos **@GeneratedValue** para indicar que es autogenerado y **GenerationType.IDENTITY** strategy se usa para mapear el valor autogenerado "id" y que pueda ser recuperado en el POJO por java
- Existen otros generadores de clave según el gestor de BD que usemos:
  - https://www.baeldung.com/hibernate-identifiers

```java
@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;
```

```cpp
CREATE TABLE PROFESOR
(
   ID_PROFESOR BIGINT NOT NULL AUTO_INCREMENT,
   NOMBRE VARCHAR (40) NOT NULL,
   NICKNAME VARCHAR (40) NOT NULL,
   APELLIDOS VARCHAR (40) NOT NULL,
   PRIMARY KEY (ID_PROFESOR)
);
```

Propiedades

@Column nos permite definir con que columnas de la tabla mapear las propiedades, podemos utilizar esta anotación sobre propiedades:

Esta anotación puede llevar atributos que todos ellos son opcionales:

- **name:** El nombre de la columna (default nombre de la propiedad)
- **unique**: establecer restricción (default false)
- **nullable**: establecer como nullable (default true).
- **insertable**: la columna será parte de la sentencia de inserción (default true)
- **updatable**: la columna será parte de la sentencia de update(default true)
- **length**: longitud(default 255)
- **precision**: precision decimal (default 0)
- **scale**: escala decimal (default 0)

```
@Column(name = "NICKNAME")
private String username;

@Column(name = "NOMBRE")
private String nombreProfesor;

@Column(name = "APELLIDOS")
private String apellidosProfesor;
```

Propiedades

Las propiedades definen que atributos del POJO se harán persistentes

- **@Transient** indica que la propiedad no se hará persistente
- **@Basic** indica que la propiedad se hará persistente. Se puede omitir. Una propiedad sin anotación se hará persistente igualmente.

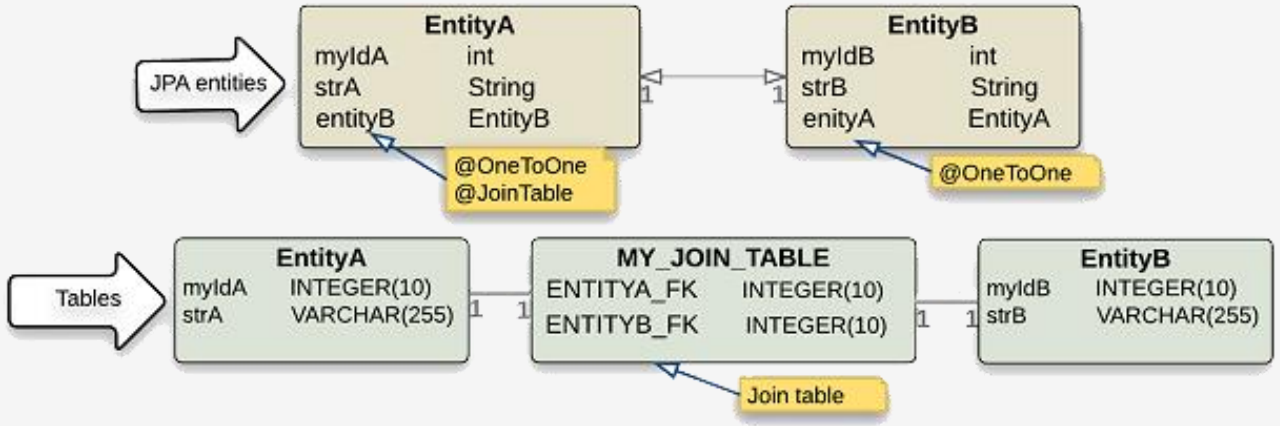
Hibernate

- **@Enumerated** la utilizaremos para indicar que un tipo enumerado se almacene como String. Si no aparece la anotación se almacena como ordinal
- **@Enumerated**(Enumtype.String)
- **@Temporal** para definir la precisión con la que almacenar datos temporales en BD (DATE, TIME o TIMESTAMP)
- **@LOB** para indicar que se haga persistente en Clob si el tipo es java.sql.Clob, Character[], char[], java.lang.String o en Blob si el tipo es java.sql.Blob, Byte[], byte[] y Serializable

```
@Column(name = "NOMBRE")
private String nombre;

@Lob
@Column(name = "IMAGEN")
private byte[] imagen;
```

Asociaciones

Uno de las mayores ventajas de empelar Hibernate es la de establecer relaciones entre entidades.

En base de datos son conocidas el uso de relaciones **1:1, 1:n o N:M**. Estas relaciones existen del mismo modo en Hibernate y es importante el conocer como mapearlas